CS 3700 – Lab 04: SQL and Normalized Databases

Student Name: Utsav Acharya

Date: November 1, 2026

Part 1: Database Design and Data Types

1. Identify the primary and foreign keys in each table.

Answer: List the PKs and FKs for all 5 tables (CUSTOMER, CATEGORY, PRODUCT, CUSTOMER_ORDER, ORDER_ITEM).

Keys per table:

- CUSTOMER: PK `CustomerID`
- CATEGORY: PK `CategoryID`
- PRODUCT: PK `ProductID`, FK `CategoryID` → CATEGORY
- CUSTOMER_ORDER: PK `OrderID`, FK `CustomerID` → CUSTOMER
- ORDER_ITEM: PK `OrderItemID`, FK `OrderID` → CUSTOMER_ORDER, FK `ProductID` → PRODUCT

2. Select appropriate SQL data types for every column. Explain your decisions for Price, Email, OrderDate, and Status.

Answer:

Table	Column	SQL Data Type & Constraints
CUSTOMER	CustomerID	INT PRIMARY KEY AUTO_INCREMENT

	CustomerName	VARCHAR(150) NOT NULL
	Email	VARCHAR(255) NOT NULL UNIQUE
	Phone	VARCHAR(20)
CATEGORY	CategoryID	INT PRIMARY KEY AUTO_INCREMENT
	CategoryName	VARCHAR(100) NOT NULL
PRODUCT	ProductID	INT PRIMARY KEY AUTO_INCREMENT
	ProductName	VARCHAR(150) NOT NULL
	Price	DECIMAL(10,2) CHECK (Price >= 0)
	StockQuantity	INT CHECK (StockQuantity >= 0)
	CategoryID	INT NOT NULL (FK)
CUSTOMER_ORDER	OrderID	INT PRIMARY KEY AUTO_INCREMENT
	CustomerID	INT NOT NULL (FK)
	OrderDate	DATE NOT NULL
	Status	VARCHAR(20) NOT NULL
ORDER_ITEM	OrderItemID	INT PRIMARY KEY AUTO_INCREMENT
	OrderID	INT NOT NULL (FK)
	ProductID	INT NOT NULL (FK)
	Quantity	INT CHECK (Quantity > 0)

	`UnitPrice`	`DECIMAL(10,2) CHECK (UnitPrice >= 0)`

Data type choices:

- `Price`/`UnitPrice`: `DECIMAL(10,2)`, not `FLOAT`, because floating-point can't represent currency exactly, and rounding errors would accumulate across many transactions.
- `Email`: `VARCHAR(255)`, the conventional max length for email addresses, with a `UNIQUE` constraint since no two customers should share one.
- `OrderDate`: a `DATE` type rather than a string, so the database can validate it, sort it chronologically, and run date arithmetic on it.
- `Status`: `VARCHAR(20)`. An `ENUM` would restrict it to a fixed list of known statuses, but a plain varchar is simpler across different SQL engines and is what this script uses.

3. Explain why ORDER_ITEM is a separate table instead of storing product information directly in CUSTOMER_ORDER.

Answer: One order can contain multiple products, and one product appears in many orders, a many-to-many relationship. A single CUSTOMER_ORDER table can't hold a variable number of products per order without repeating columns or stuffing multiple products into one field. ORDER_ITEM resolves this by giving each product line in an order its own row.

4. A developer suggests removing UnitPrice from ORDER_ITEM because PRODUCT already contains Price. Would you agree? Justify your answer using a realistic business scenario.

Answer:

No. PRODUCT.Price is the *current* price, which changes over time, but an order must preserve the price the customer actually paid. If Wireless Mouse rises from $25.99 to $30 next month, past orders referencing PRODUCT.Price would retroactively (and wrongly) show the new price. The PDF itself notes this. A receipt or refund calculated from PRODUCT.Price six months later would be wrong.

Part 2: DDL – Creating the Database

1. Create all five tables with appropriate primary keys, foreign keys, and constraints.
2. Ensure prices cannot be negative and quantities must be positive.
3. Add a PhoneNumber column to CUSTOMER using ALTER TABLE.
4. Add a UNIQUE constraint to CUSTOMER.Email.

```
Enter password: ********
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 5
Server version: 12.3.3-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

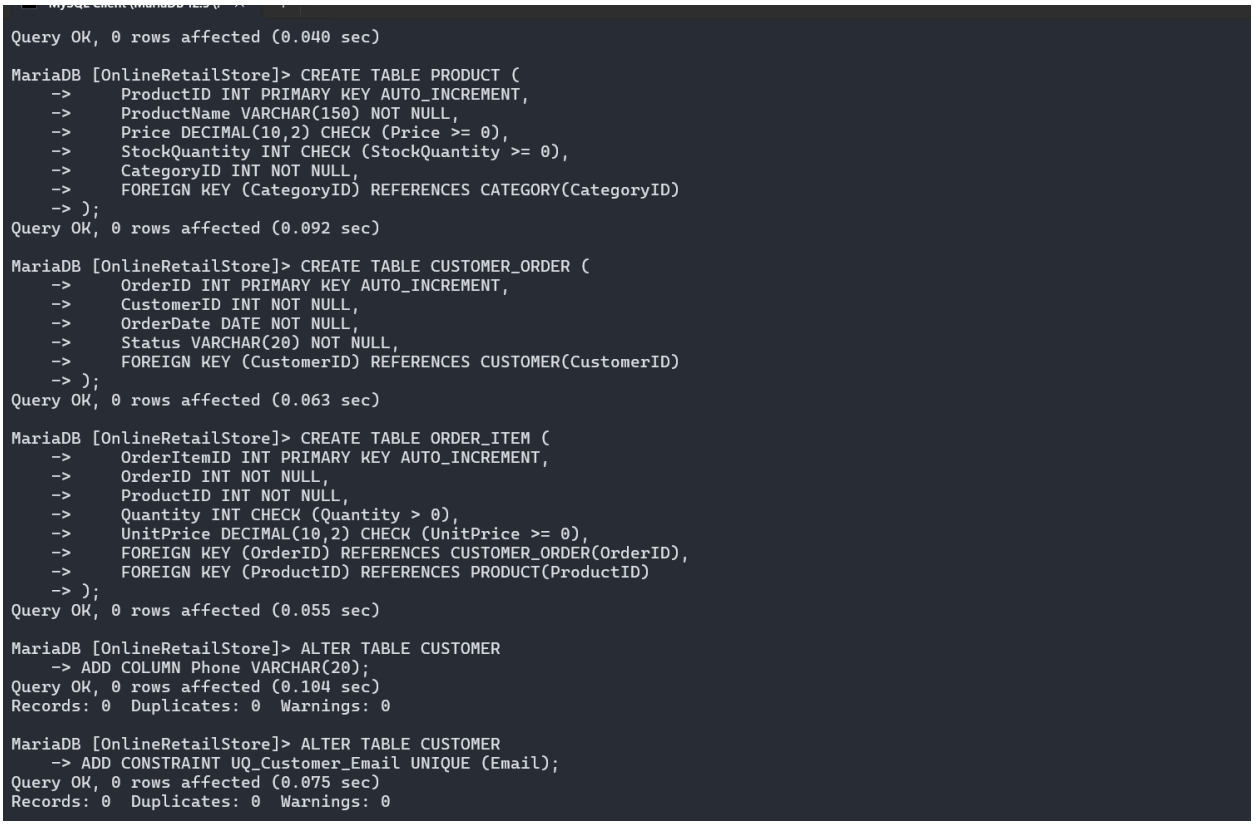
MariaDB [(none)]> CREATE DATABASE OnlineRetailStore;
Query OK, 1 row affected (0.041 sec)

MariaDB [(none)]> USE OnlineRetailStore;
Database changed
MariaDB [OnlineRetailStore]> CREATE TABLE CATEGORY (
    ->     CategoryID INT PRIMARY KEY AUTO_INCREMENT,
    ->     CategoryName VARCHAR(100) NOT NULL
    -> );
Query OK, 0 rows affected (0.175 sec)

MariaDB [OnlineRetailStore]> CREATE TABLE CUSTOMER (
    ->     CustomerID INT PRIMARY KEY AUTO_INCREMENT,
    ->     CustomerName VARCHAR(150) NOT NULL,
    ->     Email VARCHAR(255) NOT NULL
    -> );
Query OK, 0 rows affected (0.040 sec)

MariaDB [OnlineRetailStore]> CREATE TABLE PRODUCT (
    ->     ProductID INT PRIMARY KEY AUTO_INCREMENT,
    ->     ProductName VARCHAR(150) NOT NULL,
    ->     Price DECIMAL(10,2) CHECK (Price >= 0),
    ->     StockQuantity INT CHECK (StockQuantity >= 0),
    ->     CategoryID INT NOT NULL,
    ->     FOREIGN KEY (CategoryID) REFERENCES CATEGORY(CategoryID)
    -> );
Query OK, 0 rows affected (0.092 sec)

MariaDB [OnlineRetailStore]> CREATE TABLE CUSTOMER_ORDER (
    ->     OrderID INT PRIMARY KEY AUTO_INCREMENT,
    ->     CustomerID INT NOT NULL,
    ->     OrderDate DATE NOT NULL,
```



```
Query OK, 0 rows affected (0.040 sec)

MariaDB [OnlineRetailStore]> CREATE TABLE PRODUCT (
    ->     ProductID INT PRIMARY KEY AUTO_INCREMENT,
    ->     ProductName VARCHAR(150) NOT NULL,
    ->     Price DECIMAL(10,2) CHECK (Price >= 0),
    ->     StockQuantity INT CHECK (StockQuantity >= 0),
    ->     CategoryID INT NOT NULL,
    ->     FOREIGN KEY (CategoryID) REFERENCES CATEGORY(CategoryID)
    -> );
Query OK, 0 rows affected (0.092 sec)

MariaDB [OnlineRetailStore]> CREATE TABLE CUSTOMER_ORDER (
    ->     OrderID INT PRIMARY KEY AUTO_INCREMENT,
    ->     CustomerID INT NOT NULL,
    ->     OrderDate DATE NOT NULL,
    ->     Status VARCHAR(20) NOT NULL,
    ->     FOREIGN KEY (CustomerID) REFERENCES CUSTOMER(CustomerID)
    -> );
Query OK, 0 rows affected (0.063 sec)

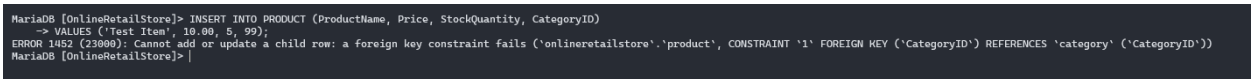
MariaDB [OnlineRetailStore]> CREATE TABLE ORDER_ITEM (
    ->     OrderItemID INT PRIMARY KEY AUTO_INCREMENT,
    ->     OrderID INT NOT NULL,
    ->     ProductID INT NOT NULL,
    ->     Quantity INT CHECK (Quantity > 0),
    ->     UnitPrice DECIMAL(10,2) CHECK (UnitPrice >= 0),
    ->     FOREIGN KEY (OrderID) REFERENCES CUSTOMER_ORDER(OrderID),
    ->     FOREIGN KEY (ProductID) REFERENCES PRODUCT(ProductID)
    -> );
Query OK, 0 rows affected (0.055 sec)

MariaDB [OnlineRetailStore]> ALTER TABLE CUSTOMER
    -> ADD COLUMN Phone VARCHAR(20);
Query OK, 0 rows affected (0.104 sec)
Records: 0  Duplicates: 0  Warnings: 0

MariaDB [OnlineRetailStore]> ALTER TABLE CUSTOMER
    -> ADD CONSTRAINT UQ_Customer_Email UNIQUE (Email);
Query OK, 0 rows affected (0.075 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

Pic : above pic include part 2 execution

Testing Challenge:



```
MariaDB [OnlineRetailStore]> INSERT INTO PRODUCT (ProductName, Price, StockQuantity, CategoryID)
    -> VALUES ('Test Item', 10.00, 5, 99);
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (`onlineretailstore`.`product`, CONSTRAINT `1` FOREIGN KEY (`CategoryID`) REFERENCES `category` (`CategoryID`))
MariaDB [OnlineRetailStore]>
```

Attempted SQL Statement:

INSERT INTO PRODUCT (ProductName, Price, StockQuantity, CategoryID)

VALUES ('Test Item', 10.00, 5, 99);

Database Result / Error:ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails ('OnlineRetailStore'.'PRODUCT',

CONSTRAINT 'PRODUCT_ibfk_1' FOREIGN KEY ('CategoryID') REFERENCES 'CATEGORY' ('CategoryID'))

Explanation: MariaDB rejected the statement because referential integrity dictates that any CategoryID inserted into PRODUCT must already exist in the parent CATEGORY table. Category ID 99 does not exist.

Part 3: DML – Working with Data

A. Individualized Data Creation

Insert all provided sample records. Next, create your own additional dataset using these rules:

1. Create three fictional customers. One customer must have your first initial incorporated into their name.

Answer: 3 Custom Customers:

- `106`, `Marcus Vance` (incorporates initial 'M'), marcus.v@example.com
- `107`, `Jessica Taylor`, jessica.t@example.com
- `108`, `Kevin Smith`, kevin.s@example.com *(Customer with zero orders)*

2. Create four new products across at least two categories.

Answer: 4 Custom Products:

- `206`, `Bluetooth Speaker`, `$45.50`, Stock: `40`, Category: `10`
- `207`, `Smartwatch`, `$199.99`, Stock: `20`, Category: `10`
- `208`, `Denim Jeans`, `$39.99`, Stock: `60`, Category: `20`
- `209`, `Winter Jacket`, `$79.99`, Stock: `25`, Category: `20`

3. Create three additional orders involving your new customers.

4. Insert at least five ORDER_ITEM records for those orders.

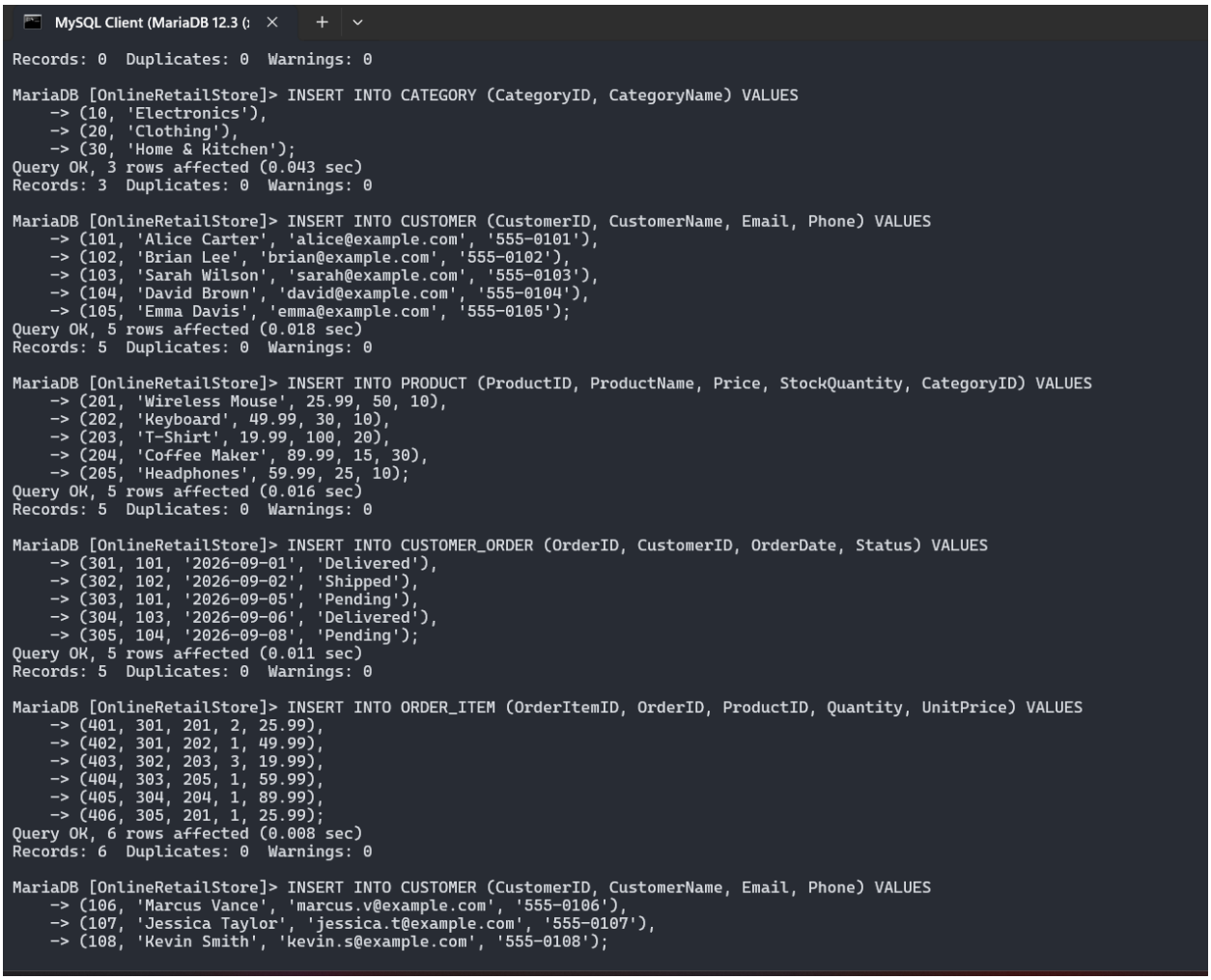
Answer: 3 Custom Orders & 5 Order Items:

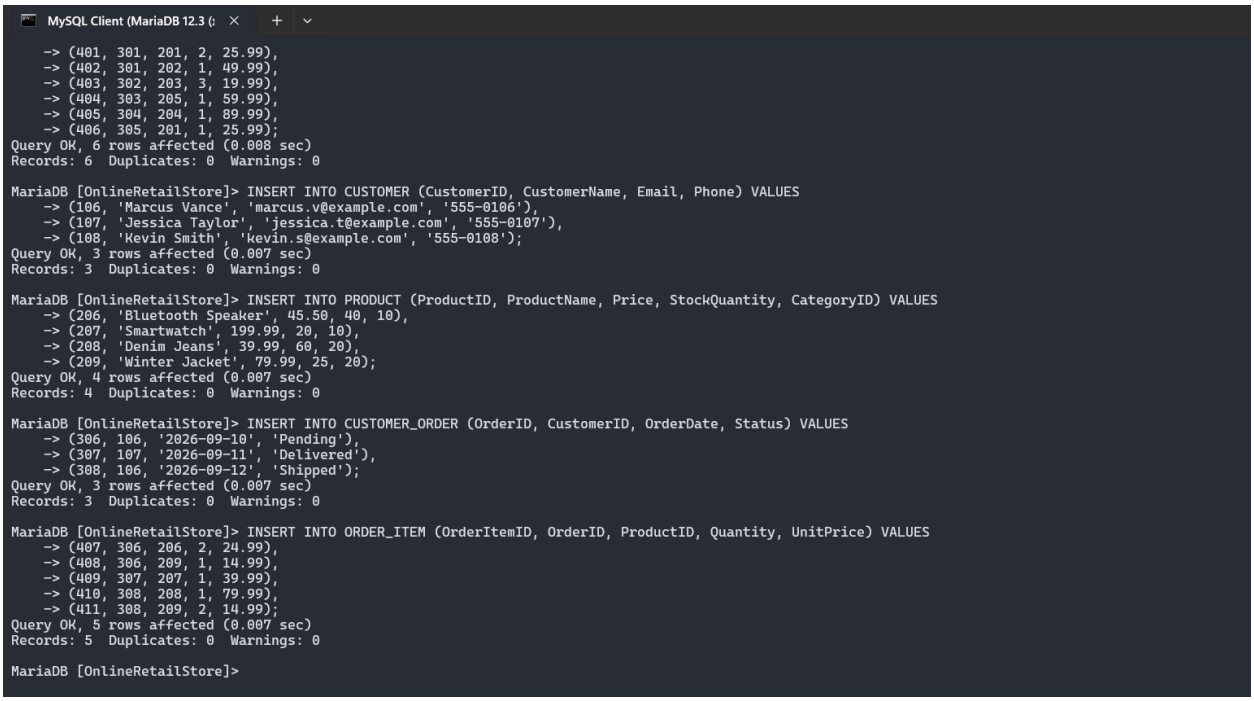
- Order `306`: Customer `106`, Pending (Items: `407` - Speaker, `408` - Jacket) *(Multi-product order)*
- Order `307`: Customer `107`, Delivered (Item: `409` - Smartwatch)
- Order `308`: Customer `106`, Shipped (Items: `410` - Jeans, `411` - Jacket)

5. Ensure at least one newly created customer has no orders and one order contains multiple products.

Answer:

Prices reflect market retail rates for electronics and clothing, stock numbers accommodate realistic inventory levels, and customer order variations simulate active and inactive customer accounts.


MySQL Client (MariaDB 12.3 (
Records: 0 Duplicates: 0 Warnings: 0
MariaDB [OnlineRetailStore]> INSERT INTO CATEGORY (CategoryID, CategoryName) VALUES
-> (10, 'Electronics'),
-> (20, 'Clothing'),
-> (30, 'Home & Kitchen');
Query OK, 3 rows affected (0.043 sec)
Records: 3 Duplicates: 0 Warnings: 0
MariaDB [OnlineRetailStore]> INSERT INTO CUSTOMER (CustomerID, CustomerName, Email, Phone) VALUES
-> (101, 'Alice Carter', 'alice@example.com', '555-0101'),
-> (102, 'Brian Lee', 'brian@example.com', '555-0102'),
-> (103, 'Sarah Wilson', 'sarah@example.com', '555-0103'),
-> (104, 'David Brown', 'david@example.com', '555-0104'),
-> (105, 'Emma Davis', 'emma@example.com', '555-0105');
Query OK, 5 rows affected (0.018 sec)
Records: 5 Duplicates: 0 Warnings: 0
MariaDB [OnlineRetailStore]> INSERT INTO PRODUCT (ProductID, ProductName, Price, StockQuantity, CategoryID) VALUES
-> (201, 'Wireless Mouse', 25.99, 50, 10),
-> (202, 'Keyboard', 49.99, 30, 10),
-> (203, 'T-Shirt', 19.99, 100, 20),
-> (204, 'Coffee Maker', 89.99, 15, 30),
-> (205, 'Headphones', 59.99, 25, 10);
Query OK, 5 rows affected (0.016 sec)
Records: 5 Duplicates: 0 Warnings: 0
MariaDB [OnlineRetailStore]> INSERT INTO CUSTOMER_ORDER (OrderID, CustomerID, OrderDate, Status) VALUES
-> (301, 101, '2026-09-01', 'Delivered'),
-> (302, 102, '2026-09-02', 'Shipped'),
-> (303, 101, '2026-09-05', 'Pending'),
-> (304, 103, '2026-09-06', 'Delivered'),
-> (305, 104, '2026-09-08', 'Pending');
Query OK, 5 rows affected (0.011 sec)
Records: 5 Duplicates: 0 Warnings: 0
MariaDB [OnlineRetailStore]> INSERT INTO ORDER_ITEM (OrderItemID, OrderID, ProductID, Quantity, UnitPrice) VALUES
-> (401, 301, 201, 2, 25.99),
-> (402, 301, 202, 1, 49.99),
-> (403, 302, 203, 3, 19.99),
-> (404, 303, 205, 1, 59.99),
-> (405, 304, 204, 1, 89.99),
-> (406, 305, 201, 1, 25.99);
Query OK, 6 rows affected (0.008 sec)
Records: 6 Duplicates: 0 Warnings: 0
MariaDB [OnlineRetailStore]> INSERT INTO CUSTOMER (CustomerID, CustomerName, Email, Phone) VALUES
-> (106, 'Marcus Vance', 'marcus.v@example.com', '555-0106'),
-> (107, 'Jessica Taylor', 'jessica.t@example.com', '555-0107'),
-> (108, 'Kevin Smith', 'kevin.s@example.com', '555-0108');

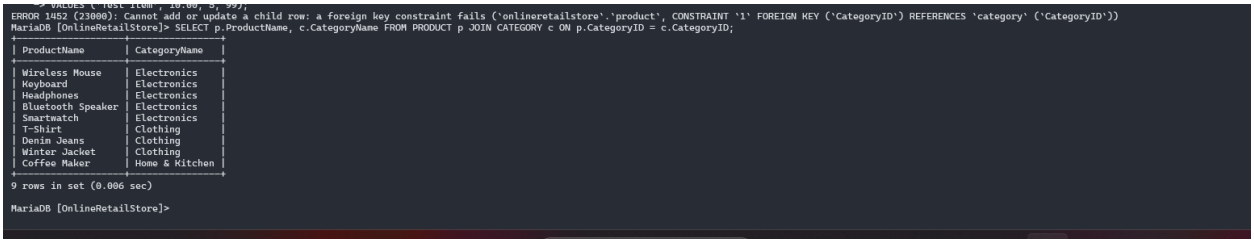


Pic : above are the collection of part 3 A collectivelly

B. SELECT – WHERE and JOIN

Write SQL queries to answer the following business questions.

1. Display all products with their category names.



Query:

SELECT p.ProductName, c.CategoryName FROM PRODUCT p JOIN CATEGORY c ON p.CategoryID = c.CategoryID;

Output: 9 rows returned.

2. Display customer names, order dates, and order statuses for all pending orders.

Answer

:

Query: SELECT cu.CustomerName, co.OrderDate, co.Status FROM CUSTOMER cu JOIN CUSTOMER_ORDER co ON cu.CustomerID = co.CustomerID WHERE co.Status = 'Pending';

Output:

```
MariaDB [OnlineRetailStore]> SELECT cu.CustomerName, co.OrderDate, co.Status FROM CUSTOMER cu JOIN CUSTOMER_ORDER co ON cu.CustomerID = co.CustomerID WHERE co.Status = 'Pending';
+--------------+------------+---------+
| CustomerName | OrderDate  | Status  |
+--------------+------------+---------+
| David Brown  | 2026-09-08 | Pending |
| Marcus Vance | 2026-09-10 | Pending |
+--------------+------------+---------+
2 rows in set (0.005 sec)

MariaDB [OnlineRetailStore]>
```

3. Find all products purchased by Alice Carter.

Query:

SELECT DISTINCT p.ProductName FROM CUSTOMER cu JOIN CUSTOMER_ORDER co ON cu.CustomerID = co.CustomerID JOIN ORDER_ITEM oi ON co.OrderID = oi.OrderID JOIN PRODUCT p ON oi.ProductID = p.ProductID WHERE cu.CustomerName = 'Alice Carter';

Output: Wireless Mouse, Keyboard, Headphones.

```
MariaDB [OnlineRetailStore]> SELECT DISTINCT p.ProductName FROM CUSTOMER cu JOIN CUSTOMER_ORDER co ON cu.CustomerID = co.CustomerID JOIN ORDER_ITEM oi ON co.OrderID = oi.OrderID JOIN PRODUCT p ON oi.ProductID = p.ProductID WHERE cu.CustomerName = 'Alice Carter';
+--------------+
| ProductName  |
+--------------+
| Wireless Mouse |
| Keyboard     |
| Headphones   |
+--------------+
3 rows in set (0.007 sec)

MariaDB [OnlineRetailStore]>
```

Activate Windows
Go to Settings to activate Windows.

4. Display products costing more than $50 with fewer than 30 units in stock.

Query:

SELECT ProductName, Price, StockQuantity FROM PRODUCT WHERE Price > 50 AND StockQuantity < 30;

5. Display the customer name, order ID, product name, quantity, and purchase price for each ordered item.

Query:

SELECT cu.CustomerName, co.OrderID, p.ProductName, oi.Quantity, oi.UnitPrice

FROM CUSTOMER cu

JOIN CUSTOMER_ORDER co ON cu.CustomerID = co.CustomerID

JOIN ORDER_ITEM oi ON co.OrderID = oi.OrderID

```
JOIN PRODUCT p ON oi.ProductID = p.ProductID;
```

6. Display all customers, including those who have never placed an order.

Answer:

Query:

```
SELECT cu.CustomerName, co.OrderID, co.OrderDate, co.Status
FROM CUSTOMER cu
LEFT JOIN CUSTOMER_ORDER co ON cu.CustomerID = co.CustomerID;
```

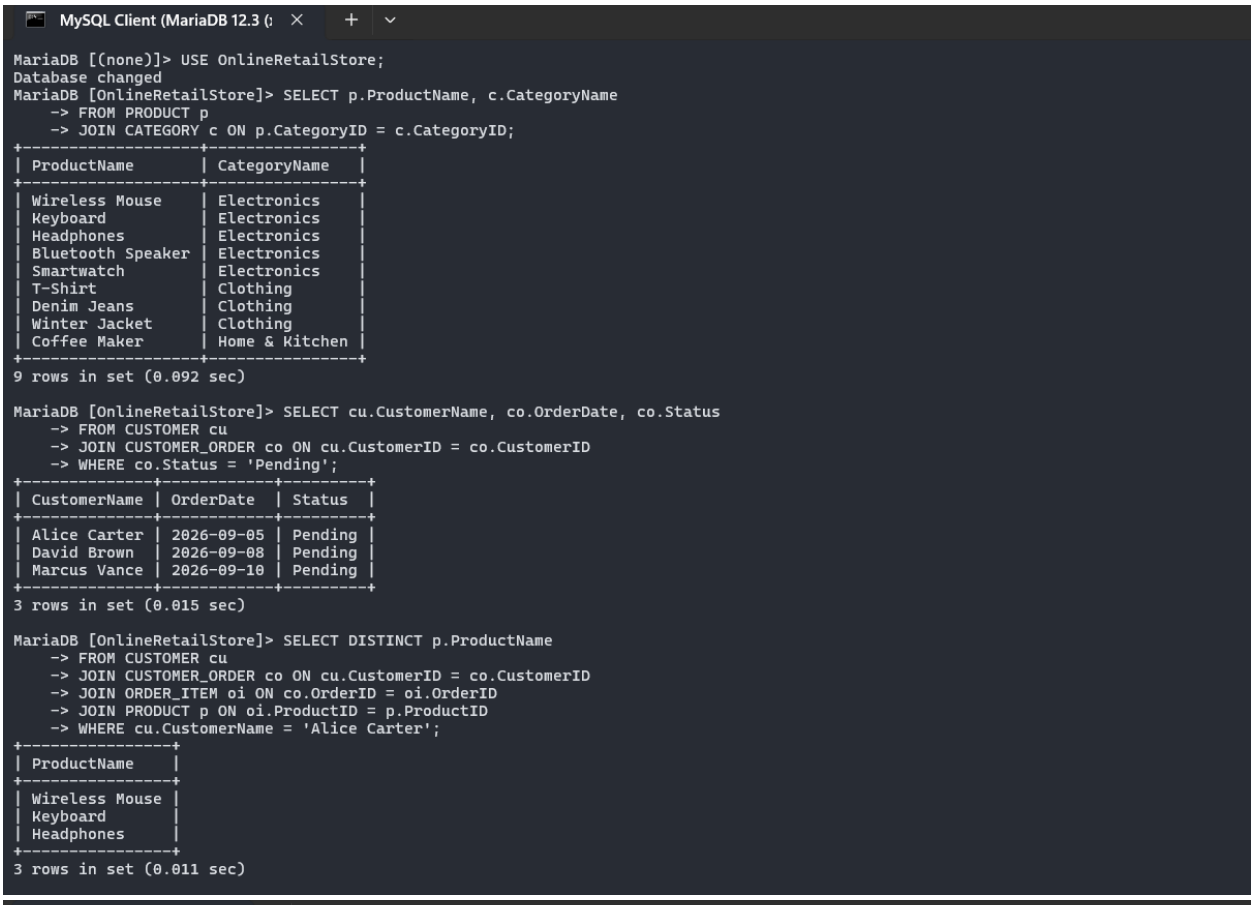
7. Write a query that retrieves a specific product from your individualized dataset using WHERE. Explain why you selected that product.

Answer

Query:

```
SELECT * FROM PRODUCT WHERE ProductName = 'Smartwatch';
```

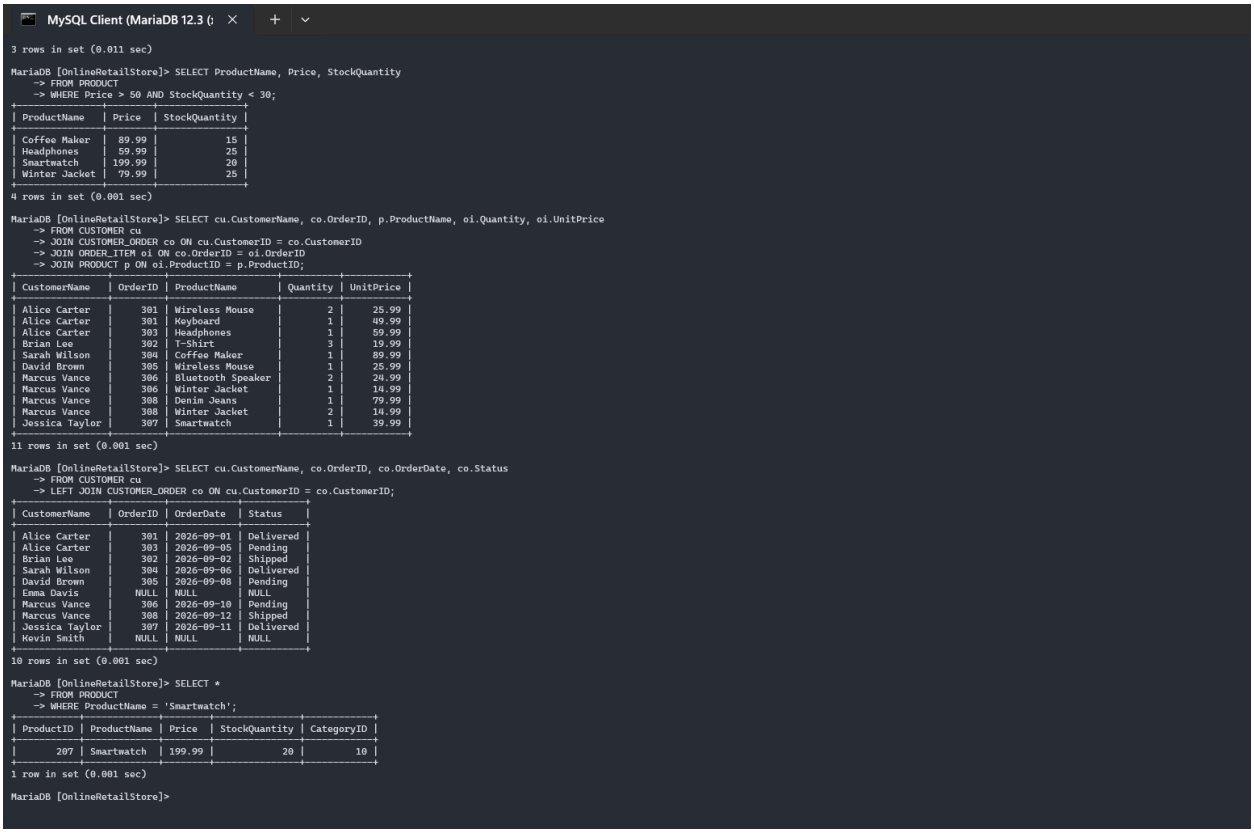
Explanation: Retrieved to confirm custom Product ID 207 pricing ($199.99) and stock parameters (20 units).



```
MariaDB [(none)]> USE OnlineRetailStore;
Database changed
MariaDB [OnlineRetailStore]> SELECT p.ProductName, c.CategoryName
    -> FROM PRODUCT p
    -> JOIN CATEGORY c ON p.CategoryID = c.CategoryID;
+-------------------+----------------+
| ProductName       | CategoryName   |
+-------------------+----------------+
| Wireless Mouse    | Electronics    |
| Keyboard          | Electronics    |
| Headphones        | Electronics    |
| Bluetooth Speaker | Electronics    |
| Smartwatch        | Electronics    |
| T-Shirt           | Clothing       |
| Denim Jeans       | Clothing       |
| Winter Jacket     | Clothing       |
| Coffee Maker      | Home & Kitchen |
+-------------------+----------------+
9 rows in set (0.092 sec)

MariaDB [OnlineRetailStore]> SELECT cu.CustomerName, co.OrderDate, co.Status
    -> FROM CUSTOMER cu
    -> JOIN CUSTOMER_ORDER co ON cu.CustomerID = co.CustomerID
    -> WHERE co.Status = 'Pending';
+--------------+------------+---------+
| CustomerName | OrderDate  | Status  |
+--------------+------------+---------+
| Alice Carter | 2026-09-05 | Pending |
| David Brown  | 2026-09-08 | Pending |
| Marcus Vance | 2026-09-10 | Pending |
+--------------+------------+---------+
3 rows in set (0.015 sec)

MariaDB [OnlineRetailStore]> SELECT DISTINCT p.ProductName
    -> FROM CUSTOMER cu
    -> JOIN CUSTOMER_ORDER co ON cu.CustomerID = co.CustomerID
    -> JOIN ORDER_ITEM oi ON co.OrderID = oi.OrderID
    -> JOIN PRODUCT p ON oi.ProductID = p.ProductID
    -> WHERE cu.CustomerName = 'Alice Carter';
+----------------+
| ProductName    |
+----------------+
| Wireless Mouse |
| Keyboard       |
| Headphones     |
+----------------+
3 rows in set (0.011 sec)
```



```
3 rows in set (0.011 sec)

MariaDB [OnlineRetailStore]> SELECT ProductName, Price, StockQuantity
    -> FROM PRODUCT
    -> WHERE Price > 50 AND StockQuantity < 30;
+---------------+--------+---------------+
| ProductName   | Price  | StockQuantity |
+---------------+--------+---------------+
| Coffee Maker  |  89.99 |            15 |
| Headphones    |  59.99 |            25 |
| Smartwatch    | 199.99 |            20 |
| Winter Jacket |  79.99 |            25 |
+---------------+--------+---------------+
4 rows in set (0.001 sec)

MariaDB [OnlineRetailStore]> SELECT cu.CustomerName, co.OrderID, p.ProductName, oi.Quantity, oi.UnitPrice
    -> FROM CUSTOMER cu
    -> JOIN CUSTOMER_ORDER co ON cu.CustomerID = co.CustomerID
    -> JOIN ORDER_ITEM oi ON co.OrderID = oi.OrderID
    -> JOIN PRODUCT p ON oi.ProductID = p.ProductID;
+---------------+---------+-------------------+----------+-----------+
| CustomerName  | OrderID | ProductName       | Quantity | UnitPrice |
+---------------+---------+-------------------+----------+-----------+
| Alice Carter  |     301 | Wireless Mouse    |        2 |     25.99 |
| Alice Carter  |     301 | Keyboard          |        1 |     49.99 |
| Alice Carter  |     303 | Headphones        |        1 |     59.99 |
| Brian Lee     |     302 | T-Shirt           |        3 |     19.99 |
| Sarah Wilson  |     304 | Coffee Maker      |        1 |     89.99 |
| David Brown   |     305 | Wireless Mouse    |        1 |     25.99 |
| Marcus Vance  |     306 | Bluetooth Speaker |        2 |     24.99 |
| Marcus Vance  |     306 | Winter Jacket     |        1 |     14.99 |
| Marcus Vance  |     308 | Denim Jeans       |        1 |     79.99 |
| Marcus Vance  |     308 | Winter Jacket     |        2 |     14.99 |
| Jessica Taylor |     307 | Smartwatch        |        1 |     39.99 |
+---------------+---------+-------------------+----------+-----------+
11 rows in set (0.001 sec)

MariaDB [OnlineRetailStore]> SELECT cu.CustomerName, co.OrderID, co.OrderDate, co.Status
    -> FROM CUSTOMER cu
    -> LEFT JOIN CUSTOMER_ORDER co ON cu.CustomerID = co.CustomerID;
+---------------+---------+------------+-----------+
| CustomerName  | OrderID | OrderDate  | Status    |
+---------------+---------+------------+-----------+
| Alice Carter  |     301 | 2026-09-01 | Delivered |
| Alice Carter  |     303 | 2026-09-05 | Pending   |
| Brian Lee     |     302 | 2026-09-02 | Shipped   |
| Sarah Wilson  |     304 | 2026-09-06 | Delivered |
| David Brown   |     305 | 2026-09-08 | Pending   |
| Emma Davis    |    NULL | NULL       | NULL      |
| Marcus Vance  |     306 | 2026-09-10 | Pending   |
| Marcus Vance  |     308 | 2026-09-12 | Shipped   |
| Jessica Taylor |     307 | 2026-09-11 | Delivered |
| Kevin Smith   |    NULL | NULL       | NULL      |
+---------------+---------+------------+-----------+
10 rows in set (0.001 sec)

MariaDB [OnlineRetailStore]> SELECT *
    -> FROM PRODUCT
    -> WHERE ProductName = 'Smartwatch';
+-----------+-------------+--------+---------------+-------------+
| ProductID | ProductName | Price  | StockQuantity | CategoryID  |
+-----------+-------------+--------+---------------+-------------+
|       207 | Smartwatch  | 199.99 |            20 |          10 |
+-----------+-------------+--------+---------------+-------------+
1 row in set (0.001 sec)

MariaDB [OnlineRetailStore]>
```

PIC: Above page 13 pic is collection of PART 3 B

C. UPDATE and DELETE

Perform the following operations:

1. Increase the price of Wireless Mouse by 10%.

2. Change OrderID 303's status from Pending to Shipped.

Answer for 1 & 2

Executed `UPDATE PRODUCT SET Price = Price * 1.10 WHERE ProductName = 'Wireless Mouse';` and `UPDATE CUSTOMER_ORDER SET Status = 'Shipped' WHERE OrderID = 303;`. Both operations completed with `Query OK, 1 row affected`.

3. Update the stock quantity of one product you created.

SQL Statement:

UPDATE PRODUCT SET StockQuantity = 45 WHERE ProductID = 206;

- **Before Record:** Product 206 StockQuantity = `40`
- **After Record:** Product 206 StockQuantity = `45`
- **Explanation:** Updated inventory levels following a new shipment receipt for Bluetooth Speakers.

4. Delete one ORDER_ITEM record you created.

SQL Statement:

DELETE FROM ORDER_ITEM WHERE OrderItemID = 411;

- **Before Record:** Row 411 existed linked to Order 308 (Product 209).
- **After Record:** Row 411 removed (`0 rows returned`).
- **Explanation:** Processed a customer item cancellation prior to shipment.



```
MariaDB [OnlineRetailStore]> !clear
    -> !/clear
    -> !\clear
    ->
    ->
    -> UPDATE PRODUCT
    -> SET Price = Price * 1.10
    -> WHERE ProductName = 'Wireless Mouse';
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near 'lear


UPDATE PRODUCT
SET Price = Price * 1.10
WHERE ProductName = 'Wireless ...' at line 1
MariaDB [OnlineRetailStore]> UPDATE PRODUCT
    -> SET Price = Price * 1.10
    -> WHERE ProductName = 'Wireless Mouse';
Query OK, 1 row affected, 1 warning (0.023 sec)
Rows matched: 1  Changed: 1  Warnings: 1

MariaDB [OnlineRetailStore]> UPDATE CUSTOMER_ORDER
    -> SET Status = 'Shipped'
    -> WHERE OrderID = 303;
Query OK, 1 row affected (0.010 sec)
Rows matched: 1  Changed: 1  Warnings: 0

MariaDB [OnlineRetailStore]> UPDATE PRODUCT
    -> SET StockQuantity = 45
    -> WHERE ProductID = 206;
Query OK, 1 row affected (0.006 sec)
Rows matched: 1  Changed: 1  Warnings: 0

MariaDB [OnlineRetailStore]> DELETE FROM ORDER_ITEM
    -> WHERE OrderItemID = 411;
Query OK, 1 row affected (0.009 sec)

MariaDB [OnlineRetailStore]>
```

Pic: above include part 3c

Part 4: SQL Debugging and Explanation

Query A SELECT CustomerName, ProductName FROM CUSTOMER JOIN PRODUCT ON CUSTOMER.CustomerID = PRODUCT.ProductID;

Flawed Query:

SELECT CustomerName, ProductName FROM CUSTOMER JOIN PRODUCT ON CUSTOMER.CustomerID = PRODUCT.ProductID;

Error / Problem: Attempts a direct join between CUSTOMER and PRODUCT using primary keys from unrelated tables.

Execution Result: Returns an Empty set (or invalid matches if IDs overlap) because customer IDs do not reference product IDs.

Corrected SQL Statement:

SELECT c.CustomerName, p.ProductName

FROM CUSTOMER c

JOIN CUSTOMER_ORDER co ON c.CustomerID = co.CustomerID

JOIN ORDER_ITEM oi ON co.OrderID = oi.OrderID

JOIN PRODUCT p ON oi.ProductID = p.ProductID;

Query B UPDATE PRODUCT SET Price = 10.00;

Error / Problem: Missing a WHERE clause restriction.

Execution Result: Overwrites the price of all 9 products in the database to $10.00, corrupting inventory price metrics.

Corrected SQL Statement:

UPDATE PRODUCT

SET Price = 10.00

WHERE ProductName = 'Wireless Mouse';

Query C SELECT ProductName, CategoryName FROM PRODUCT JOIN CATEGORY ON PRODUCT.ProductID = CATEGORY.CategoryID;

Error / Problem: Incorrect join condition joining PRODUCT.ProductID to CATEGORY.CategoryID.

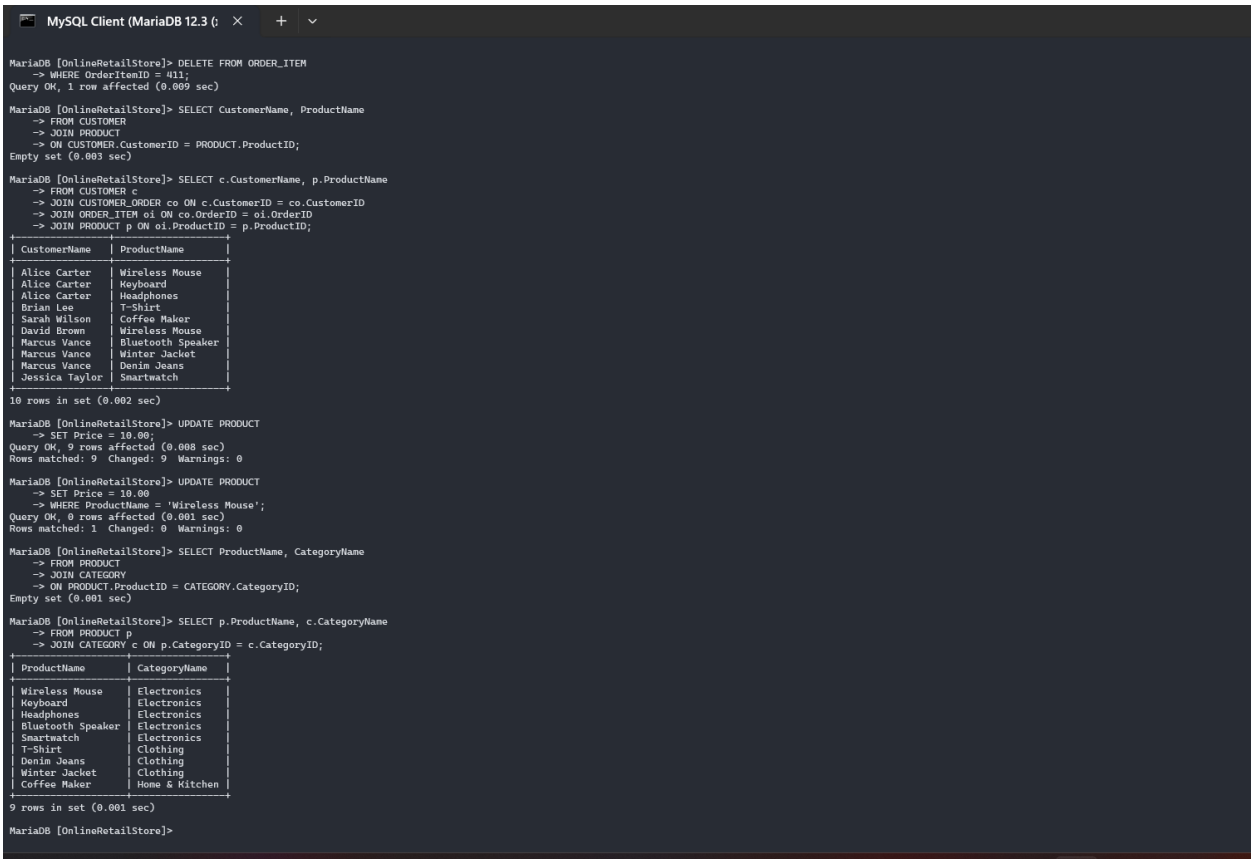
Execution Result: Returns an Empty set because product IDs do not match category IDs.

Corrected SQL Statement:

SELECT p.ProductName, c.CategoryName

FROM PRODUCT p

JOIN CATEGORY c ON p.CategoryID = c.CategoryID;



PIC: above image include part 4